The Vault

Ejercicio Integrador — resolución paso a paso

72.07 — Protocolos de Comunicación · ITBA
Objetivo: **entrar a The Vault** (SSH en `6.9.4.20`, sin internet dentro del laboratorio)
Resuelto el 07/11/2026

Índice

1. Enunciado y diagrama de red
2. Paso 0 — Conexión al laboratorio (SSH + túnel Cloudflare)
3. Paso 1 — Reconocimiento de dónde caímos
4. Paso 2 — Escaneo de los routers (nmap) y el challenge `/start`
5. Desafío 1 — Gateway de pampero (`/juialepan`)
6. Desafío 2 — Serial SOA por DNS (`/ketylanuchi`)
7. Desafío 3 — Negociación de contenido (`/uruuruluru`)
8. Desafío 4 — Token por email (`/papopepoparapapapapiparapopepo`)
9. Desafío 5 — Validación sin estado (`/dalequevamooo`)
10. Paso final — Ruta a secret-network, la clave SSH y entrar a The Vault
11. Resumen de todo lo usado

1. Enunciado y diagrama de red

Nos dan acceso a un laboratorio militar escondido en el Ártico. El servidor **“The Vault”** guarda secretos de estado. La persona que lo administra tomó varias medidas de seguridad para que no podamos entrar, y **adentro del laboratorio no hay internet**. El objetivo es uno solo: **entrar a The Vault**.

Instrucciones de conexión que da la cátedra

- Usuario SSH = la parte de tu mail de ITBA antes de la `@` (ej. `kbuttowski@itba.edu.ar` → usuario `kbuttowski`).
- Contraseña SSH = tu legajo.
- El servidor SSH **no** tiene IP pública: se expone por un **túnel de Cloudflare**, así que hay que darle un *ProxyCommand* al cliente SSH usando `cloudflared` .

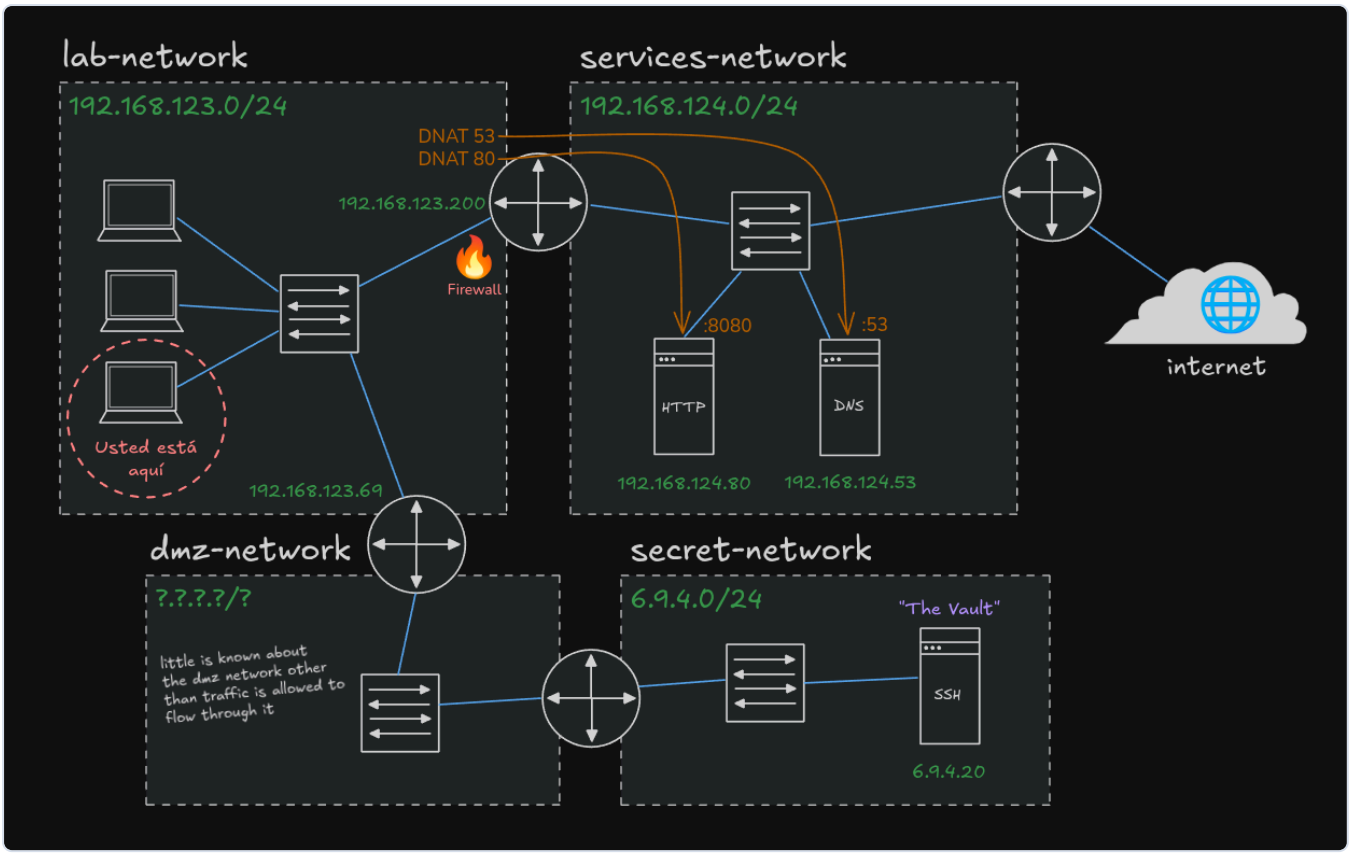


Diagrama de red entregado por el comandante. "Usted está aquí" = lab-network.

Cómo leer el diagrama (clave para toda la resolución)

Red	Subred	Qué hay / qué importa
lab-network	`192.168.123.0/24`	Donde caemos nosotros. Router/firewall en `.200` (hace **DNAT** de los puertos 80 y 53 hacia services-network) y otro router en `.69` que conecta hacia la **dmz**.
services-network	`192.168.124.0/24`	Servidor HTTP (`.80:8080`), servidor DNS (`.53:53`) y **la única salida a internet**. No la tocamos directo: llegamos vía el DNAT del `.200`.
dmz-network	`?.?.?.?/?`	"Se sabe poco, solo que el tráfico pasa por ahí." Es el puente hacia secret-network.
secret-network	`6.9.4.0/24`	**The Vault**: servidor SSH en `6.9.4.20`. Es nuestro destino final.

> **Lo que el diagrama nos adelanta:** para llegar a The Vault (`6.9.4.0/24`) el tráfico tiene que pasar por el router `.69` → dmz → secret-network. Y cuando necesitemos hablar HTTP o DNS, el router `.200` ya nos redirige (DNAT) a los servidores reales de services-network. Guardá esto: lo vamos a usar literalmente al final.

2. Paso 0 — Conexión al laboratorio

El SSH del lab no tiene IP pública; sale por un túnel de Cloudflare. La forma más simple es levantar `cloudflared` dentro de un contenedor Docker descartable y usarlo como *ProxyCommand* de SSH.

Instalar Docker y bajar la imagen

```
docker pull cloudflare/cloudflared
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
```

Error típico la primera vez: `permission denied` al socket de Docker. Pasa porque recién te agregaste al grupo `docker` (`usermod -aG docker $USER`) pero **esa terminal todavía no tiene el grupo cargado**. Se arregla sin reiniciar sesión con:

```
newgrp docker   # recarga los grupos en la terminal actual
```

Después `docker run --rm hello-world` ya funciona.

Conectarse por SSH a través del túnel

```
ssh -o ProxyCommand="docker run --rm -i cloudflare/cloudflared access ssh --hostname %h" \
    jdileomuntaner@protolab.thomasmiz.me
```

Parte del comando	Qué hace
`-o ProxyCommand="..."`	En vez de conectar por TCP directo, SSH ejecuta ese comando y usa su entrada/salida como "cable" hacia el servidor.
`docker run --rm -i`	Levanta un contenedor; `--rm` lo borra al terminar, `-i` mantiene stdin abierto (SSH necesita hablar por ahí).
`cloudflared access ssh --hostname %h`	`cloudflared` abre el túnel de Cloudflare hacia el host. `%h` lo reemplaza SSH por el hostname destino.
`jdileomuntaner@...`	Usuario = parte antes de la `@` del mail de ITBA. La contraseña pedida es el legajo.

Resultado: entramos como **root** a un contenedor llamado `ctf-lab-jdileomuntaner`. Ese contenedor es nuestra "máquina dentro del lab" (el nodo *"usted está aquí"* del diagrama).

3. Paso 1 — Reconocimiento: ¿dónde caímos?

Antes de atacar nada, hay que entender dónde estamos parados: qué IP tenemos, a qué redes llegamos y qué DNS usamos.

```
root@ctf-lab-jdileomuntaner:/# hostname
ctf-lab-jdileomuntaner
root@ctf-lab-jdileomuntaner:/# whoami
root
root@ctf-lab-jdileomuntaner:/# id
uid=0(root) gid=0(root) groups=0(root)

root@ctf-lab-jdileomuntaner:/# ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if66        UP             192.168.123.4/24

root@ctf-lab-jdileomuntaner:/# ip route
192.168.123.0/24 dev eth0 proto kernel scope link src 192.168.123.4

root@ctf-lab-jdileomuntaner:/# cat /etc/resolv.conf
# Generated by Docker Engine.
nameserver 127.0.0.11
search lan
options edns0 trust-ad ndots:0
```

Comando	Para qué sirve	Qué aprendimos
`hostname` / `whoami` / `id`	Quién soy y con qué permisos.	Somos **root** en un contenedor (por eso no hace falta `sudo`).
`ip -br a`	Ver IPs de las interfaces (`-br` = formato breve).	Tenemos `192.168.123.4/24` → estamos en **lab-network**, como decía el diagrama.
`ip route`	Ver la tabla de ruteo.	**Una sola línea** y **sin default gateway**: solo alcanzamos nuestra propia red. No hay salida a internet ni a otras redes... *todavía*.
`cat /etc/resolv.conf`	Ver qué DNS usa la máquina.	Apunta al resolver interno de Docker (`127.0.0.11`), **no** al DNS del lab. Las pistas del lab hay que pedírselas a mano al DNS del `.200`.

> **¿Por qué importa no tener ruta default?** El default gateway es la ruta que usa la máquina para *"cualquier destino que no conozco"*. Si no hay, todo lo que no sea `192.168.123.0/24` responde `Network is unreachable`. Esto nos va a morder más adelante con `6.9.4.20`, y ahí lo vamos a resolver agregando una ruta a mano.

4. Paso 2 — Escanear los routers y encontrar el challenge

Solo alcanzamos lab-network, así que vemos qué ofrecen los dos routers que conocemos: `.200` (hacia services) y `.69` (hacia la dmz). `nmap -sV` escanea puertos e intenta identificar el servicio/versión de cada uno.

```
root@ctf-lab-jdileomuntaner:/# nmap -sV 192.168.123.200
Nmap scan report for ...-servicenet-router... (192.168.123.200)
PORT   STATE SERVICE VERSION
80/tcp open  http
... (nmap no reconoce el server pero captura el cuerpo:)
<h2>Bienvenido!</h2>...<a href="/start">/start</a>

root@ctf-lab-jdileomuntaner:/# nmap -sV 192.168.123.69
All 1000 scanned ports on ...-dmznet-router... (192.168.123.69) are in ignored states.
Not shown: 1000 closed tcp ports (reset)
```

Opción de nmap	Qué hace
`-sV`	Service/Version detection: no solo dice qué puerto está abierto, sino que intenta averiguar qué programa corre y su versión.

Conclusión: el `.200` tiene el puerto **80 abierto** con un HTTP que nos invita a `/start`. El `.69` no expone nada propio (es solo un router: lo vamos a *atravesar*, no atacar). Vamos al HTTP.

Primer contacto con el HTTP

```
root@ctf-lab-jdileomuntaner:/# curl -i http://192.168.123.200
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
...
<h2>Bienvenido!</h2><p>Este desafío es imposible, mejor rendite así no perdés tiempo.</p>
<p>Si querés proceder, accedé a <a href="/start">/start</a></p>
```

> **¿Por qué `/start`?** Porque la propia página de bienvenida nos lo dice: trae un `<a href="/start">`. En HTTP, un enlace así es "el próximo recurso al que ir". No lo adivinamos: lo leímos en la respuesta. La opción `-i` de `curl` es la que nos deja ver **headers + cuerpo** (por eso la usamos en todos los pasos).

```
root@ctf-lab-jdileomuntaner:/# curl -i 192.168.123.200/start
HTTP/1.1 200 OK
...
No te respondo nada sin un header "Tiki: Taka"!
```

El server exige un header propio. Lo agregamos con `-H`:

```
root@ctf-lab-jdileomuntaner:/# curl -i -H "Tiki: Taka" 192.168.123.200/start
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
...
Para responder cada pregunta de este challenge, deberás concatenar a la URL un /response,
realizar un POST a dicho recurso y enviar un JSON (indicando al servidor que eso es un JSON)
un documento con la respuesta: {"rta": "entendido"}.
Entendido?
```

El protocolo del challenge (se repite en las 5 preguntas)

1. **GET** a la ruta → te devuelve la **pregunta**.
2. Resolvés la pregunta con herramientas de red.
3. **POST** a `ruta/response` con cuerpo JSON `{"rta": "tu respuesta"}` y header `Content-Type: application/json` → te devuelve la **siguiente ruta**.

Confirmamos que entendimos haciendo el primer POST:

```
root@ctf-lab-jdileomuntaner:/# curl -i -X POST -H "Tiki: Taka" \
     -H "Content-Type: application/json" -d '{"rta": "entendido"}' \
     192.168.123.200/start/response
Excelente!
Procedemos a /juialepan para tu primer desafío.
```

Opción de curl	Qué hace
`-X POST`	Usa el método HTTP POST (por defecto curl usa GET).
`-H "Header: valor"`	Agrega un header. `Content-Type: application/json` le avisa al server que el cuerpo es JSON.
`-d '...'`	El cuerpo (data) de la petición. Acá, el JSON con la respuesta.

> **Observación que hicimos en el camino:** el header `Tiki: Taka` **solo** era obligatorio en `/start`. En las rutas siguientes lo probamos con y sin, y daba igual → era específico de esa ruta, no global. Lo dejamos de mandar.

5. Desafío 1 — Gateway de pampero (`/juialepan`)

```
root@ctf-lab-jdileomuntaner:/# curl -i 192.168.123.200/juialepan
## PRIMERA PREGUNTA
¿Cuál es el gateway de pampero para llegar a la IP 1.1.1.1?
Su respuesta debe ser una dirección IP.
```

La pregunta es sobre **pampero** (una máquina de la facultad, distinta del lab). En pampero preguntamos qué gateway usaría para llegar a `1.1.1.1`:

```
[jdileomuntaner@pampero ~]$ ip route get 1.1.1.1
1.1.1.1 via 10.16.1.254 dev enp1s0 src 10.16.1.100 uid 63051
    cache
```

> **¿Por qué `ip route get` y no `ip route` a secas?** `ip route get IP` le pregunta al kernel *"si tuviera que mandar un paquete a esta IP concreta, ¿por dónde y por qué gateway saldría?"*. Resuelve la decisión real de ruteo (incluyendo la ruta default) en una línea. El campo `via` es exactamente el gateway. Respuesta: `10.16.1.254`.

```
root@ctf-lab-jdileomuntaner:/# curl -i -X POST -H "Content-Type: application/json" \
     -d '{"rta": "10.16.1.254"}' 192.168.123.200/juialepan/response
Muy bien!
Procedemos a /ketylanuchi para tu segunda pregunta.
```

6. Desafío 2 — Serial SOA por DNS (`/ketylanuchi`)

```
root@ctf-lab-jdileomuntaner:/# curl -i 192.168.123.200/ketylanuchi
## SEGUNDA PREGUNTA: El router que expone este servidor HTTP también expone un servidor DNS.
¿Cuál es el SERIAL de www.google.com según este?
```

La pista es clave: *"el mismo router expone un DNS"*. O sea, el `.200` también atiende en el puerto 53 (por el DNAT del diagrama). Le preguntamos el registro **SOA** de `www.google.com` **a ese DNS puntual**:

```
root@ctf-lab-jdileomuntaner:/# dig @192.168.123.200 www.google.com SOA
;; ANSWER SECTION:
www.google.com.   3600    IN  CNAME   google.com.
google.com.       604800  IN  SOA     ns.google.com. noveaslostxt.google.com. 74757169 69 420
666 69
```

Parte de `dig`	Qué significa
`@192.168.123.200`	Preguntarle **a ese servidor DNS** y no al del sistema. Imprescindible: el resolv.conf apuntaba al DNS de Docker.
`www.google.com`	El nombre a consultar.
`SOA`	El tipo de registro: *Start Of Authority*, que trae los metadatos de la zona.

¿Dónde está el serial? El registro SOA tiene el formato: `<ns-primario> <mail-admin> <SERIAL> <refresh> <retry> <expire> <minimum>` . El **serial** es el primer número después de los dos nombres → `74757169` . (Ojo: el server respondió un CNAME a `google.com.` y el SOA es de esa zona, que es lo correcto.)

```
root@ctf-lab-jdileomuntaner:/# curl -i -X POST -H "Content-Type: application/json" \
     -d '{"rta": "74757169"}' 192.168.123.200/ketylanuchi/response
Acertado.
Procedemos a /urururuluru para la tercer pregunta.
```

7. Desafío 3 — Negociación de contenido (`/ururuluru`)

```
root@ctf-lab-jdileomuntaner:/# curl -i 192.168.123.200/ururuluru
## TERCERA PREGUNTA: ¿Cuál es el contenido EN LENGUAJE ESPAÑOL del recurso /turulu?
```

Pedimos `/turulu` normal y vemos qué devuelve:

```
root@ctf-lab-jdileomuntaner:/# curl -i 192.168.123.200/turulu
HTTP/1.1 200 OK
Content-Language: en
X-Sugerencia: https://developer.mozilla.org/.../Accept-Language
...
<h3>The cat is under the table</h3>
```

La pista está en los headers. El server responde `Content-Language: en` y hasta nos deja un `X-Sugerencia` apuntando a la doc de `Accept-Language`. Eso es **negociación de contenido**: el mismo recurso tiene varias representaciones (idiomas) y el cliente pide la que quiere con el header `Accept-Language`. Pedimos español:

```
root@ctf-lab-jdileomuntaner:/# curl -i -H "Accept-Language: es" 192.168.123.200/turulu
HTTP/1.1 200 OK
Content-Language: es
...
Hoy aprendimos a negociar contenido sobre HTTP
```

```
root@ctf-lab-jdileomuntaner:/# curl -i -X POST -H "Content-Type: application/json" \
     -d '{"rta": "Hoy aprendimos a negociar contenido sobre HTTP"}' \
     192.168.123.200/ururuluru/response
Bien. Aprender es importante.
Procedemos a /papopepoparapapapapiparapopepo para la cuarta pregunta.
```

8. Desafío 4 — Token por email (`/papopepoparapapapapiparapopepo`)

```
root@ctf-lab-jdileomuntaner:/# curl -i 192.168.123.200/papopepoparapapapapiparapopepo
## CUARTA PREGUNTA
El endpoint POST /maketoken?email=usuario@dominio es capaz de enviar mails secretos,
pero no a cualquier dirección!
Para recibir un mail, debe entrar a https://temp-mail.org/en/ ...
En este se encontrará un token generado únicamente para usted. Dicho token es la respuesta.
```

Primero probamos con un dominio cualquiera, y falla:

```
root@ctf-lab-jdileomuntaner:/# curl -i -X POST "192.168.123.200/maketoken?
email=test@example.com"
HTTP/1.1 500 Internal Server Error
Perdón, hubo un problema enviando el mail a test@example.com :(
```

> **¿Por qué falla?** Recordá del Paso 1: el contenedor **no tiene salida a internet** (sin ruta default). El server que manda los mails sí tiene que salir, y solo lo logra hacia direcciones que pueda alcanzar. Una casilla temporal de `temp-mail.org` sí funciona. Sacamos una (`yojopip415@drtall.com`) y la usamos:

```
root@ctf-lab-jdileomuntaner:/# curl -i -X POST "192.168.123.200/maketoken?
email=yojopip415@drtall.com"
Checkeá tu casilla de SPAM :)
Nota: si no te llega el mail, pegale a /maketoken/damearchivoplis?email=usuario@dominio
```

El mail que llegó



El mail de "El Guardasecretos". Lleno de emojis y colores para distraer.



Sigue el mail: "NUNCA LO VAS A ENCONTRAR". Puro bluff.

La trampa. El mail es `multipart/alternative` : trae una parte **HTML** (la vistosa, con texto blanco sobre fondo blanco escondido "entre las letras") y una parte **text/plain** mucho más simple. La parte HTML es un **señuelo**. El token real está en la parte de texto plano, en base64. Para verlo alcanza con mirar el **source** del mail / la parte text/plain, sin pelear con el HTML.

En la parte de texto plano aparecía este bloque en base64:

```
UGVybyBxIGJ1ZW5vIHEgc295IGVzY29uZGllbmRvIG1lbnNhamVzIHhEeGQKCkVsIHRva2Vu
IGVzOiBZMkZ0YVd4dlBXTmhiV2xzYnp0eWRHRTljM1J5YVc1blgyVnhkV0ZzY3c9PQoKU2Fs
dWRvcwoK
```

Lo decodificamos. Se puede sin `echo` / `printf` /heredoc, pasando el string con `<<<` (here-string):

```
base64 -d <<< 'UGVybyBxIGJ1ZW5vIHEgc295...(todo el bloque)...'
Pero q bueno q soy escondiendo mensajes xDxd

El token es: Y2FtaWxvPWNhbWlsbztydGE9c3RyaW5nX2VxdWFscw==

Saludos
```

Comando	Qué hace
`base64 -d`	`-d` = decode. Convierte el texto base64 de vuelta a su contenido original.
`<<< 'texto'`	*Here-string* de bash: le manda ese texto como stdin al comando, sin usar `echo` ni archivos.

Detalle importante. Lo que devolvió **es el token**, y resulta que el token en sí **también** es base64 (`Y2FtaWxv...`). Pero la pregunta pedía **el token**, no su contenido: hay que mandar el string tal cual vino, **sin** decodificar la segunda capa.

```
root@ctf-lab-jdileomuntaner:/# curl -i -X POST -H "Content-Type: application/json" \
     -d '{"rta": "Y2FtaWxvPWNhbWlsbztydGE9c3RyaW5nX2VxdWFscw=="}' \
     192.168.123.200/papopepoparapapapapiparapopepo/response
Muy bien. (Espero que hayas entendido lo que hiciste)
Procedemos a /dalequevamooo para tu última pregunta >:)
```

9. Desafío 5 — Validación sin estado (`/dalequevamooo`)

```
root@ctf-lab-jdileomuntaner:/# curl -i 192.168.123.200/dalequevamooo
## ÚLTIMA PREGUNTA!
Sabiendo que este servidor no tiene estado (y por ende no se acuerda qué tokens emitió),
¿Cómo hace para saber que el token que recién ingresaste no fue inventado?
La respuesta a esta pregunta se encuentra en el mismísimo token.
```

Si decodificamos la **segunda capa** del token (lo que antes no hacía falta mandar):

```
base64 -d <<< 'Y2FtaWxvPWNhbWlsbztydGE9c3RyaW5nX2VxdWFscw=='
camilo=camilo;rta=string_equals
```

Qué muestra el token. El token es **autocontenido**: adentro trae el valor esperado y **cómo** compararlo (`rta=string_equals` → "comparar como igualdad de strings"). El server no necesita guardar nada: recibe el token, lo lee y compara. El problema (consigna B de la teoría) es que **no está firmado**, así que cualquiera podría fabricarse un token válido. Pero la pregunta pedía **el mecanismo**, no una explicación.

Lección práctica. Primero mandé una explicación en prosa y el server la rechazó:

```
-d '{"rta": "El token trae adentro el valor esperado y como compararlo..."}'
HTTP/1.1 400 Bad Request — Respuesta incorrecta.
```

Estos endpoints esperan el **valor exacto**, no un párrafo. La respuesta era literalmente el nombre del mecanismo: `string_equals`.

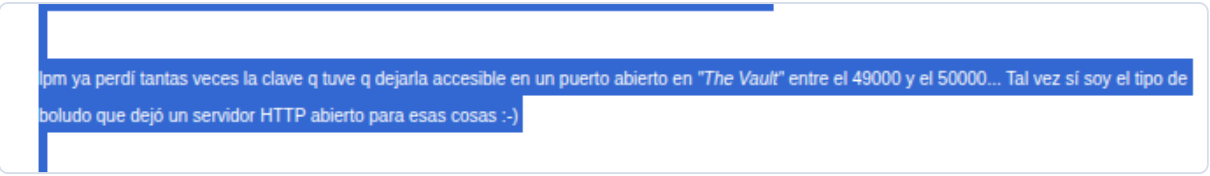
```
root@ctf-lab-jdileomuntaner:/# curl -i -X POST -H "Content-Type: application/json" \
     -d '{"rta": "string_equals"}' 192.168.123.200/dalequevamooo/response
Felicidades!
La respuesta es correcta; "string_equals".
Esta también resulta ser **la contraseña de la clave SSH** de el servidor The Vault...
(Si aún no sabés dónde encontrar esta clave, volvé al mensaje de email y leé entre las
líneas...)
;-)
```

Dos premios en una. (1) `string_equals` es también la **passphrase** de la clave SSH de The Vault. (2) El server nos manda a releer el email "entre líneas" → el texto blanco escondido que habíamos descartado como señuelo en realidad tenía una pista.

10. Paso final — Entrar a The Vault

La pista escondida en el email

Seleccionando el texto blanco-sobre-blanco del mail (o mirando el source) aparecía esta línea:



El texto oculto, revelado al seleccionarlo.

> *"...ya perdí tantas veces la clave q tuve q dejarla accesible en un puerto abierto en 'The Vault' entre el **49000 y el 50000**... tal vez sí soy el tipo de boludo que dejó un servidor HTTP abierto para esas cosas :-)"*

Traducción: en The Vault (`6.9.4.0/24`) hay un **servidor HTTP** escuchando en algún puerto entre 49000 y 50000 que sirve la clave privada SSH.

Problema: no llegamos a secret-network

```
root@ctf-lab-jdileomuntaner:/# ping -c 2 6.9.4.20
ping: connect: Network is unreachable
```

> **¿Por qué?** Otra vez el problema del Paso 1: no tenemos ruta hacia `6.9.4.0/24` . El diagrama dice que a secret-network se llega **atravesando la dmz**, y el router hacia la dmz es el `192.168.123.69` . Entonces le decimos al kernel: *"para ir a `6.9.4.0/24` , mandá por `.69` "*.

```
root@ctf-lab-jdileomuntaner:/# ip route add 6.9.4.0/24 via 192.168.123.69
```

Parte del comando	Qué significa
`ip route add`	Agregar una ruta estática nueva a la tabla de ruteo.
`6.9.4.0/24`	La red destino que queremos alcanzar (toda secret-network).
`via 192.168.123.69`	El próximo salto (gateway) por el que mandar esos paquetes: el router hacia la dmz.

```
root@ctf-lab-jdileomuntaner:/# ping -c 2 6.9.4.20
64 bytes from 6.9.4.20: icmp_seq=1 ttl=62 time=0.071 ms
64 bytes from 6.9.4.20: icmp_seq=2 ttl=62 time=0.074 ms

root@ctf-lab-jdileomuntaner:/# ip route
6.9.4.0/24 via 192.168.123.69 dev eth0
192.168.123.0/24 dev eth0 proto kernel scope link src 192.168.123.4
```

> **El `ttl=62` confirma el camino.** El TTL arranca normalmente en 64 y baja 1 por cada router que atraviesa. 64 − 62 = **2 saltos** → pasó por el router `.69` y por el router de secret-network, justo como dibuja el diagrama (lab → dmz → secret).

Buscar el servidor HTTP en el rango 49000–50000

```
root@ctf-lab-jdileomuntaner:/# nmap -p 49000-50000 6.9.4.20
PORT      STATE SERVICE
49681/tcp open  unknown
```

`-p 49000-50000` limita el escaneo a ese rango de puertos (el que indicaba la pista). Hay uno abierto: **49681**.

```
root@ctf-lab-jdileomuntaner:/# curl -i 6.9.4.20:49681
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.12.3
...
<h1>Directory listing for /</h1>
  <li><a href="cat.jpg">cat.jpg</a></li>
  <li><a href="IMPORTANTE_NOMBRE_DE_USUARIO.txt">IMPORTANTE_NOMBRE_DE_USUARIO.txt</a></li>
  <li><a href="supersecretkey">supersecretkey</a></li>
```

> **Jackpot.** Es el `http.server` de Python (`SimpleHTTP`), que cuando no hay `index.html` muestra el **listado del directorio**. Dejó expuestos: el usuario SSH y la clave privada. Los bajamos con `curl -o` (`-o archivo` = guardar la respuesta en ese archivo):

```
root@ctf-lab-jdileomuntaner:/# curl -o usuario.txt
http://6.9.4.20:49681/IMPORTANTE_NOMBRE_DE_USUARIO.txt
root@ctf-lab-jdileomuntaner:/# curl -o supersecretkey  http://6.9.4.20:49681/supersecretkey

root@ctf-lab-jdileomuntaner:/# cat usuario.txt
IMPORTANTE!!!
El nombre de usuario de tu servidor SSH es "bububu"
NO TE OLVIDES
```

Preparar la clave y entrar

La clave privada (`supersecretkey`) está protegida con passphrase (su header interno dice `bcrypt`). La passphrase ya la teníamos: `string_equals` , del Desafío 5.

```
root@ctf-lab-jdileomuntaner:/# chmod 600 supersecretkey             # SSH rechaza claves con
permisos abiertos
root@ctf-lab-jdileomuntaner:/# ssh-keygen -y -P 'string_equals' -f supersecretkey   #
verificar passphrase sin conectar
ssh-ed25519 AAAA...   # imprime la pública → la passphrase es correcta
```

Comando / opción	Qué hace
`chmod 600`	Deja la clave legible/escribible solo por su dueño. SSH se niega a usar una clave privada si otros la pueden leer.
`ssh-keygen -y`	Deriva e imprime la clave **pública** a partir de la privada. Si la passphrase está bien, imprime la pública; si no, da error. Sirve para **verificar sin conectarse**.
`-P 'string_equals'`	La passphrase de la clave.
`-f supersecretkey`	El archivo de clave privada a usar.

```
root@ctf-lab-jdileomuntaner:/# ssh -i supersecretkey -o IdentitiesOnly=yes bububu@6.9.4.20
# passphrase: string_equals
# ¡Adentro de The Vault! Objetivo cumplido.
```

Opción de ssh	Qué hace
`-i supersecretkey`	Usar esta clave privada para autenticar.
`-o IdentitiesOnly=yes`	Usar **solo** la clave indicada con `-i` y no ofrecer otras claves del agente (evita que el server corte por demasiados intentos).
`bububu@6.9.4.20`	Usuario `bububu` (del archivo que bajamos) en The Vault.

> ✅ **Objetivo cumplido: entramos a The Vault.**

11. Resumen de todo lo usado

Cadena de la solución

1. Conexión por SSH + túnel Cloudflare (`cloudflared` en Docker).
2. Reconocimiento (`ip -br a`, `ip route`, `resolv.conf`): estamos aislados en lab-network.
3. `nmap` → HTTP en `.200` → challenge en `/start` (header `Tiki: Taka`).
4. 5 desafíos: gateway (`ip route get`) → SOA (`dig @`) → idioma (`Accept-Language`) → token por email (base64) → mecanismo `string_equals`.
5. Ruta estática a secret-network (`ip route add ... via .69`) → `nmap` del rango → HTTP de Python que filtra usuario + clave.
6. SSH con la clave (passphrase = `string_equals`) como `bububu` → **The Vault**.

Conceptos de la materia que entraron

Tema	Dónde apareció
Ruteo (gateway, tabla de rutas, rutas estáticas, TTL)	Paso 1, Desafío 1, Paso final
DNS (registro SOA, consultar un server puntual con `dig @`)	Desafío 2
HTTP: métodos, headers, status codes, negociación de contenido	Paso 2 y Desafíos 1–4
Email: `multipart/alternative`, parte HTML vs texto plano, base64	Desafío 4
Validación sin estado / token autocontenido y sin firma	Desafío 5
SSH: clave privada, passphrase vs permisos, `IdentitiesOnly`, túnel/ProxyCommand	Paso 0 y Paso final
Escaneo de servicios (`nmap -sV`, `-p rango`)	Paso 2 y Paso final

Comandos clave (chuleta)

```
ip -br a                                  # mis IPs
ip route                                  # mi tabla de rutas
ip route get 1.1.1.1                      # por qué gateway saldría a esa IP
ip route add 6.9.4.0/24 via 192.168.123.69 # ruta estática a otra red
nmap -sV HOST                             # puertos + servicio/versión
nmap -p 49000-50000 HOST                  # escanear un rango de puertos
dig @192.168.123.200 www.google.com SOA   # consultar un DNS puntual
curl -i URL                               # ver headers + cuerpo
curl -i -H "Accept-Language: es" URL       # negociar idioma
curl -i -X POST -H "Content-Type: application/json" -d '{"rta":"..."}' URL
curl -o archivo URL                       # guardar la respuesta a disco
base64 -d <<< 'TEXTO'                      # decodificar base64 sin echo/printf
chmod 600 clave                           # permisos que SSH exige
ssh-keygen -y -P 'pass' -f clave           # verificar passphrase sin conectar
ssh -i clave -o IdentitiesOnly=yes user@host
```